

Progetto ASD 2026: Algoritmi per Grafi di Autonomous Systems

Niccolò Marchesi

Agosto 2026

Moduli Software e Interazioni

Il software viene suddiviso in tre moduli principali:

Modulo di Caricamento: Interamente contenuto all'interno di una funzione `void` dedicata (`caricaGrafoDaFile`), questo modulo ha lo scopo di estrarre le sequenze numeriche degli AS dai dump BGP e le inietta dinamicamente nella struttura dati del Grafo passata per riferimento.

Specifiche

1. **Input:** Un flusso di dati proveniente da un file testuale locale, il cui percorso è fornito tramite argomento a riga di comando (`argv[1]`). Il file contiene record BGP; ciascuna riga rappresenta un singolo cammino espresso come sequenza logica di identificatori (AS ID) separati da caratteri testuali.
2. **Output:** Nessun valore di ritorno esplicito (`void`). Estrae le coppie di vertici adiacenti (u, v) e invoca i metodi del Grafo per l'inserimento effettivo.
3. **Specifiche funzionali:**
 - Prima dell'invocazione del modulo, il flusso di controllo principale (`main`) ispeziona il numero di argomenti passati a riga di comando (`argc`). Se il parametro è inferiore a 2, viene innescata una terminazione del programma notificando l'assenza del file sorgente, prevenendo a monte errori di inizializzazione.
 - Apertura del file passato per argomento tramite la classe `ifstream`.
 - Attraverso il metodo `is_open()` si verifica se l'apertura è andata a buon fine. In caso di fallimento, l'algoritmo esegue un *early-exit* logico per prevenire crash di sistema.
 - Lettura sequenziale del file riga per riga tramite il costrutto `while (getline(file, line))`, dove la funzione `getline` legge i caratteri dello stream uno ad uno fino a che non incontra il carattere speciale di fine riga, salva la stringa appena letta dentro la variabile `line` e restituisce `True` se questa operazione è andata a buon fine.
 - Trasformazione della stringa `line` in un flusso di dati tramite l'oggetto `stringstream ss(line)` (dove `ss` è il nome assegnato al flusso temporaneo).
 - Utilizzando l'operatore di estrazione `>>` si ricavano le prime due parole dal flusso. Questa estrazione viene eseguita solo se ci sono almeno due parole disponibili. (Nota: Le sequenze degli AS sono sempre il secondo elemento di ogni riga, pertanto il blocco di dati successivo viene volontariamente ignorato per ottimizzare i tempi).

- Sostituzione dinamica dei delimitatori testuali | con spazi bianchi nella seconda parola (`sequenza_as`) tramite l'algoritmo `replace`, preparando la stringa per la successiva estrazione numerica diretta.
- Usando nuovamente un oggetto `stringstream ss_numeri(sequenza_as)`, si trasforma la seconda parola modificata in un nuovo stream per leggere la sequenza degli AS.
- Inizializzazione delle variabili intere u e v . Queste sono utilizzate per mantenere in memoria due numeri consecutivi durante la lettura. La variabile u viene inizializzata a -1 per indicare che non è ancora stata completata la lettura della prima coppia.
- Lettura sequenziale dei numeri tramite l'operatore `»` all'interno del ciclo `while (ss_numeri » v)`, salvando il valore estratto in v . Successivamente si verifica che u sia diverso da -1 e che u e v siano tra loro diversi (per prevenire i self-loop). Se le condizioni sono soddisfatte, l'arco (u, v) viene inserito nel Grafo. Infine, si pone $u = v$ per passare alla coppia di valori successiva.
- Chiusura del file al termine della scansione tramite il metodo `file.close()`.

4. Librerie Standard e Strutture dati utilizzate:

- `<fstream>`: Fornisce le interfacce per le operazioni di Input/Output sul file system. In questo modulo è indispensabile per l'oggetto `ifstream`, che crea il "ponte" fisico tra il programma e il file salvato sul disco rigido.
- `<string>`: Implementa la classe standard `string`. È essenziale per la gestione dinamica, sicura ed efficiente della memoria quando si alloca la variabile testuale `line` che ospita temporaneamente la riga letta.
- `<sstream>`: Contiene la classe `stringstream`. Permette di prendere una normale stringa statica e trasformarla in un flusso di dati, abilitando l'estrazione parola per parola o numero per numero tramite l'operatore `»`, automatizzando il salto degli spazi vuoti.
- `<algorithm>`: Fornisce un vastissimo set di algoritmi matematici e di utilità fortemente ottimizzati per le prestazioni. Nel modulo è stata inclusa unicamente per poter invocare `replace`, che consente di sostituire tutti i caratteri | con spazi bianchi alterando la stringa direttamente sul posto, senza dover creare copie aggiuntive in memoria.

5. Complessità attesa:

- Il costo asintotico è lineare $O(N)$, dove N rappresenta la dimensione totale in byte del file. L'input viene letto e processato esattamente una sola volta dall'inizio alla fine, rendendo i tempi proporzionali esclusivamente alla grandezza del dump.

6. **Dipendenze e Accoppiamento:** Il modulo non dipende da altri sottomoduli. Agisce esclusivamente come sorgente dati per il Modulo Grafo.

Modulo Grafo: Rappresenta il nucleo di memorizzazione del progetto. Gestisce l'allocazione dinamica dei vertici e degli archi in memoria, implementando un'interfaccia stabile per l'aggiornamento delle frequenze e la risoluzione delle collisioni.

Specifiche

1. **Input:** Riceve singoli input sotto forma di coppie di identificatori (u, v) direttamente dal Modulo di Caricamento. (Nota: il modulo assume che l'eliminazione dei self-loop sia già stata eseguita prima della chiamata).

2. **Output:** Mantiene e aggiorna una struttura dati in memoria che rappresenta il grafo $G = (V, E, W)$ orientato e pesato. Espone metodi di accesso rapido per la navigazione delle adiacenze e il recupero dei pesi, essenziali per i successivi algoritmi di visita.

3. **Specifiche funzionali e Strutture utilizzate:**

- All'inizio vengono stabilite le entità logiche di base: **struct Arco** definisce il singolo collegamento, è formato dall'ID del nodo destinazione e dal peso dell'arco (ovvero la frequenza con cui compare).

struct Nodo rappresenta il singolo Autonomous System, contiene il proprio ID numerico e un **vector<Arco>** che funge da lista di adiacenza del nodo.

- Viene introdotta la classe **UniversalHash** per mappare gli AS ID sparsi, traducendoli in indici per il successivo salvataggio nella Tabella Hash. Questa implementa la funzione matematica:

$$h(x) = (ax + b) \bmod p \quad (1)$$

Dove a, b sono coefficienti random e p un numero primo. La classe è stata progettata nel seguente modo: nello Stato Privato (**private**) sono inseriti i coefficienti a, b e il numero primo p , mantenuti inaccessibili al resto del programma.

Nell'Interfaccia Pubblica (**public**) ci sono solo i servizi necessari. Il costruttore estrae casualmente (**rand()**) i coefficienti $a \in [1, p-1]$ e $b \in [0, p-1]$ all'avvio. Questa casualità garantisce un'ottima distribuzione delle chiavi, evitando che sequenze di dati sfortunate creino troppe collisioni nello stesso scompartimento. Infine, viene esposto il metodo **hash(long long x)** che esegue la trasformazione effettiva della chiave ($h(x)$) senza rivelare i parametri segreti usati per il calcolo.

- La struttura principale è la classe **GrafoHashTable**. Questa gestisce l'inserimento dei dati implementando una Tabella Hash con Hashing universale. Le strutture principali della classe sono:

Nel **private** vengono inseriti:

- * La dimensione m della Tabella Hash.
- * Un **vector<vector<Nodo>> table** che rappresenta la memoria della Tabella Hash. Il vettore esterno modella gli m scompartimenti, mentre quello interno implementa le liste di concatenazione per risolvere le collisioni.
- * L'Oggetto delegato **hasher** della classe **UniversalHash**. La classe **Grafo** non esegue calcoli matematici complessi al proprio interno, ma li "delega" al motore **hasher**.
- * La funzione **calcolaHash(int x)** per calcolare l'indice h di un nodo x . Questa prende il risultato di $h(x)$ della classe **UniversalHash**, vi applica l'operatore modulo m , corregge gli eventuali risultati negativi sommando m ad h e ritorna l'indice h definitivo. Questa operazione è la funzione di hash universale.

Nel **public** la struttura principale è il **Costruttore**, che alloca preventivamente la dimensione totale m della Tabella Hash e avvia la creazione dell'oggetto **UniversalHash**, passandogli il numero primo fisso $p = 998244353$ (scelto appositamente molto grande per minimizzare le collisioni).

- Una volta predisposta l'infrastruttura della Tabella Hash, viene implementata la funzione **get_nodo(int nodo_id)** (Metodo Privato): Funge da motore per il recupero o la creazione dinamica di un nodo il cui AS ID è **nodo_id**. Calcola l'indice h tramite **calcolaHash** e scandisce il relativo scompartimento **table[h]**. Se il nodo esiste lo restituisce, altrimenti lo alloca tramite **table[h].push_back({nodo_id, vector<Arco>()})**

(ovviamente `vector<Arco>()` è vuoto). Si noti che la funzione restituisce il nodo per riferimento (`Nodo&`), permettendo modifiche dirette all'oggetto senza creare inutili copie in memoria.

- Successivamente viene implementata la funzione `void aggiungi_arco(int u, int v)` (Metodo Pubblico): Rappresenta il punto di accesso utilizzato per l'inserimento degli archi nel Grafo. Prende in input due nodi u, v e richiama `get_nodo(u)` per ottenere il nodo sorgente, dopodiché scandisce la sua lista di adiacenza: se il nodo destinazione v è già presente, incrementa la frequenza dell'arco ed esce dalla funzione; se la scansione termina senza successo, crea un nuovo arco verso v con frequenza iniziale impostata a 1 nella lista di adiacenza di u tramite `.push_back({v,1})`. Le due funzioni `get_nodo` e `aggiungi_arco` sono i metodi che si occupano della costruzione effettiva del Grafo.
- Una volta che il Modulo di Caricamento ha estratto con successo la coppia di nodi u e v dal file testuale, esegue direttamente la chiamata `grafo.aggiungi_arco(u, v)`. È esattamente attraverso questa istruzione che i due moduli comunicano: il primo estrae i dati in sequenza, il secondo li riceve e li organizza. In questo modo il grafo viene fisicamente costruito passo dopo passo.
- **Inizializzazione nel main:** All'interno del `main`, viene creato l'oggetto `grafo` della classe `GrafoHashTable` con dimensione iniziale di $m = 100000$. Immediatamente dopo, viene costruito il grafo invocando la funzione `caricaGrafoDaFile(argv[1], grafo)`, passando come argomenti il percorso del file di testo (estratto dalla riga di comando) e il riferimento all'oggetto `grafo` appena creato. Questa è la funzione che contiene il Modulo di Caricamento.

4. Librerie Standard utilizzate:

- `<vector>`: Indispensabile per la gestione dinamica della memoria, sia per implementare gli scompartimenti della Tabella Hash, sia per le liste di adiacenza all'interno dei singoli nodi.
- `<random>`: Fornisce l'algoritmo `rand()` per l'estrazione pseudo-casuale dei coefficienti matematici a e b necessari per l'Hashing Universale.
- `<ctime>`: Fondamentale per interagire con l'orologio di sistema. Viene utilizzata per impostare il "seme" (seed) iniziale del generatore di numeri casuali (tramite riga di comando `srand(time(NULL))` nel `main`), garantendo che ad ogni singola esecuzione del programma la Tabella Hash utilizzi parametri realmente imprevedibili.

5. **Complessità attesa:** Grazie alle proprietà matematiche dell'hashing universale ed a un dimensionamento iniziale adeguato della tabella (m), le collisioni sono minimizzate. Le operazioni di ricerca, inserimento e aggiornamento di un arco avvengono in tempo ammortizzato costante $O(1)$.

6. **Dipendenze e Accoppiamento:** Agisce da strato intermedio. Dipende esclusivamente dai dati forniti dal Modulo di Caricamento ed agisce come base per il Modulo Algoritmico.

Modulo Algoritmico (Cammini Minimax): Questo modulo rappresenta il cuore logico del progetto. Poiché il grafo in analisi è strettamente orientato, il modulo implementa una variante dell'algoritmo di Dijkstra per individuare i cammini minimi.

A differenza del metodo standard — in cui il peso per raggiungere ciascun nodo dalla sorgente s viene pre-inizializzato a $+\infty$ e nessun nodo possiede inizialmente un predecessore — si è implementata un'inizializzazione dinamica al fine di eliminare i pesanti costi computazionali

e di memoria legati a queste assegnazioni. Tutto ciò è reso possibile dall'implementazione di un'apposita Tabella Hash che funge da tracciatore, memorizzando dinamicamente i costi dei cammini e i predecessori esclusivamente quando i nodi vengono effettivamente analizzati. Inoltre è stata adottata una popolazione dinamica della Coda di Priorità (*Lazy Dijkstra*). Questo approccio, che prevede l'inserimento dei nodi man mano che vengono scoperti invece di inserirli tutti all'inizio, è utilizzato per gestire identificatori altamente sparsi come gli AS ID. Infine, l'ultima differenza con il modello standard è nella selezione del cammino: anziché valutare la classica somma dei pesi, viene implementata la variante Dijkstra Minimax.

Specifiche

1. **Input:** L'identificatore (AS ID) di un nodo sorgente s e di un nodo destinazione d .
2. **Output:** Il cammino ottimo da s verso d , calcolato secondo la metrica Minimax, restituito come **vector** dei nodi attraversati per raggiungere la destinazione.
3. **Specifiche funzionali e Strutture utilizzate:**
 - Inizializzazione della `struct InfoCammino`. Questa definisce lo stato temporaneo di un nodo durante la visita. Contiene l'ID del nodo, il costo attuale del cammino per raggiungerlo (a partire dal nodo sorgente s) e l'AS ID del suo predecessore.
 - Creazione della classe `TabellaTracciamento` come Hash Table Custom. Funge da "taccuino" dell'algoritmo, anch'esso basato su Hashing Universale. Come nel caso della classe `GrafoHashTable`, le strutture principali sono:
Nel `private` viene inizializzata la dimensione m della Tabella Hash, un `vector<vector<InfoCammino>> table` che rappresenta la memoria della Tabella Hash con la stessa gestione delle collisioni precedente, l'oggetto delegato `hasher` della classe `UniversalHash`, e la funzione `calcolaHash(int x)` per calcolare l'indice h di un nodo x nella stessa maniera di prima.
Nel `public` la struttura principale è il `Costruttore` che è implementato analogamente a quello presente in `GrafoHashTable`.
 - Sempre nel `public` di `TabellaTracciamento` viene aggiunto il metodo `get_info(int nodo_id)`. Questo funziona in modo molto simile a `get_nodo`: calcola l'indice h tramite `calcolaHash` e scandisce il relativo scompartimento `table[h]`. Se il nodo è già stato registrato nella tabella lo restituisce, altrimenti gli assegna un costo pari a $+\infty$ e un predecessore nullo (-1) per poi aggiungerlo nella Tabella Hash tramite `table[h].push_back({nodo_id, INF, -1})`. Anche in questo caso si noti che la funzione restituisce le informazioni per riferimento (`InfoCammino&`), evitando copie inutili.
 - La ricerca vera e propria è all'interno del metodo pubblico `calcolaCamminoMinimax(int nodo_sorgente, int nodo_destinazione)`, appartenente alla classe principale `GrafoHashTable`. Questa funzione funge da vero e proprio motore: appena chiamata, crea la propria `TabellaTracciamento` locale (chiamata taccuino) che verrà distrutta a fine esecuzione; la sua dimensione m viene posta a 100000.
 - L'esecuzione dell'algoritmo impiega una Coda di Priorità `pq (priority_queue<pair<int, int>>`, dove il primo intero rappresenta il costo, il secondo l'ID) configurata come *Min-Heap* tramite `greater<pair<int, int>>` per estrarre sempre il nodo con il costo minore.
 - Viene inserito il nodo sorgente all'interno del taccuino appena creato tramite `get_info(nodo_sorgente)`, successivamente il costo viene inizializzato a 0 (il cammino da s in se stesso pesa 0) e come predecessore viene posto s stesso.

- **Popolamento dinamico (Lazy Dijkstra):** La coda viene inizializzata inserendo unicamente il nodo sorgente s (con costo 0 e predecessore se stesso). I nodi successivi vengono inseriti dinamicamente durante l'esplorazione.
- **Estrazione Iterativa e Scarto Duplicati:** Fintanto che `pq` non è vuota, a ogni ciclo viene preso il primo elemento dalla coda (che corrisponde al nodo con costo minore). Il suo ID viene assegnato alla variabile u (`int u = pq.top().second`), viene inizializzata la variabile `costo_attuale` per memorizzare il costo assegnato a u al momento del suo inserimento in coda (`int costo_attuale = pq.top().first`), e infine l'elemento viene rimosso dalla coda (`pq.pop()`). Se il nodo estratto coincide con la destinazione d , l'algoritmo si interrompe immediatamente (*Early Exit*). Successivamente, poiché l'approccio *Lazy Dijkstra* reinserisce i nodi nella coda ogni volta che trova un percorso migliore, è presente un controllo che individua e scarta istantaneamente i duplicati obsoleti estratti dalla coda. Il funzionamento è il seguente: si recuperano tramite `get_info` le informazioni del nodo estratto u ; se il `costo_attuale` risulta maggiore del costo attualmente associato al nodo u nel taccuino (`info_u.costo`), l'algoritmo scarta il nodo e passa direttamente all'iterazione successiva.
- **Fase di Aggiornamento (Variante Minimax):** Si recuperano le informazioni del nodo u tramite `get_nodo(u)` e per ogni nodo v nella lista di adiacenza di u , viene inizializzata la variabile `peso_arco` che rappresenta il peso dell'arco (u, v) e la variabile `info_v` per recuperare le informazioni di v dal taccuino. Si verifica se il passaggio attraverso u garantisce un percorso più efficiente. Invece della classica somma, il costo è dato dall'equazione:

$$\text{candidato_peso} = \max(\text{info_u.costo}, \text{peso_arco}) \quad (2)$$

Se questo `candidato_peso` risulta strettamente minore del costo attualmente registrato per v nella `TabellaTracciamento`, il costo di v diventa `candidato_peso`, si imposta u come nuovo predecessore di v e si spinge la nuova coppia nella Coda di Priorità (`pq.push({candidato_peso, v})`).

- **Ricostruzione del Percorso:** Una volta terminato il ciclo, si ricostruisce il percorso finale a ritroso usando i predecessori dei nodi, partendo dalla destinazione d . Si inizializza un vettore di interi `percorso_finale`. Vengono recuperate dal taccuino le informazioni di d per una verifica di raggiungibilità: se il predecessore di d è rimasto -1 , significa che non esiste un percorso da s a d e il modulo restituisce il vettore vuoto. Altrimenti, si inizializza la variabile `nodo_corrente` pari a `nodo_destinazione` e si avvia un ciclo `while` che itera a ritroso finché non si raggiunge il `nodo_sorgente`. Ad ogni iterazione, l'ID del nodo corrente viene memorizzato nel vettore (`.push_back(nodo_corrente)`) e la variabile `nodo_corrente` viene aggiornata con l'ID del suo predecessore (`taccuino.get_info(nodo_corrente).predecessore`). Dopo aver inserito il nodo sorgente s , poiché la sequenza risulta invertita, l'ordine viene rovesciato tramite la funzione standard `reverse` prima di restituire il vettore.
- **Esecuzione nel main:** All'interno del `main`, l'esecuzione dell'algoritmo è posta in un ciclo interattivo. Il programma richiede all'utente l'inserimento degli AS ID per il nodo sorgente (il quale se posto pari a -1 fa terminare il programma) e per il nodo destinazione (validando tramite un ciclo interno che quest'ultimo sia strettamente diverso dal sorgente). Successivamente, viene innescata la ricerca invocando il metodo `grafo.calcolaCamminoMinimax(nodo_sorgente, nodo_destinazione)` dal quale ricaviamo il vettore contenente il percorso. Infine, il programma esegue un

controllo sull'output: se il vettore risulta vuoto, notifica all'utente l'assenza di un percorso (o l'inesistenza dei nodi immessi); in caso contrario, procede alla stampa a schermo della sequenza dei nodi, separandoli visivamente per indicare la direzione del cammino (->).

4. Librerie Standard utilizzate:

- `<queue>` e `<functional>`: Lavorano in sinergia per la gestione della Coda di Priorità. `<queue>` fornisce la struttura dati `priority_queue`, mentre `<functional>` mette a disposizione il funtore logico `greater`, che è indispensabile per invertire il comportamento di default del C++ (da Max-Heap a Min-Heap) e garantire l'estrazione del nodo con il costo minore.
- `<limits>`: Viene utilizzata per generare un valore di "infinito", all'inizio del codice viene creata una costante `const int INF = numeric_limits<int>::max()` assegnato come costo iniziale a tutti i nodi appena scoperti per garantire che qualsiasi percorso reale risulti strettamente minore.
- `<algorithm>`: Utilizzata nella fase finale per richiamare la funzione `reverse`. Questo permette di capovolgere il vettore contenente il percorso estratto a ritroso, senza dover scrivere cicli di scambio manuali.
- `<vector>`: Struttura portante per l'intera gestione dinamica della memoria, impiegata sia nella `TabellaTracciamento`, sia per la memorizzazione e restituzione del `percorso_finale`.

5. **Complessità attesa:** Le operazioni di lettura e aggiornamento dei pesi nella `TabellaTracciamento` richiedono un tempo atteso costante $O(1)$, grazie all'impiego dell'Hashing universale. Il costo dominante rimane pertanto l'estrazione e l'inserimento nella Coda di Priorità (Min-Heap), che garantisce tempi logaritmici. La complessità temporale attesa nel caso peggiore si conferma quindi pari a $O(|E| \log |V|)$.

Nota: l'algoritmo di Dijkstra viene eseguito a ogni singola richiesta. Per ottimizzare i tempi, l'algoritmo si ferma non appena il nodo destinazione d viene estratto definitivamente dalla coda di priorità.

6. **Dipendenze e Accoppiamento:** Modulo completamente autonomo a livello di stato, ma interfacciato in sola lettura al *Modulo Grafo* tramite la funzione `get_nodo(u)`, utilizzata esclusivamente per recuperare le liste di adiacenza originali.

Modulo Opzionale

Questo modulo ha lo scopo di riportare:

- Il numero di nodi e di archi del grafo.
- La distribuzione delle frequenze sugli archi.
- Il costo ottimo di un cammino Minimax da s a d .
- Il numero di cammini di costo Minimax uguale al valore ottimo.
- I tempi di esecuzione degli algoritmi.

Conteggio del numero dei nodi e degli archi del Grafo: Il calcolo viene effettuato al termine del caricamento del Grafo, richiamando due metodi pubblici appositi implementati all'interno della classe `GrafoHashTable`:

- Metodo `contaNodi()`: Calcola il numero totale dei vertici $|V|$. Il metodo effettua una rapida scansione delle m celle della Tabella Hash (il `vector` esterno) e, per ciascuna di esse, sfrutta la funzione standard `.size()` sul `vector` interno (che gestisce le collisioni) per ottenerne istantaneamente la dimensione. Sommando questi valori ad una variabile temporanea `count` inizializzata a 0, si ricava la cardinalità esatta dei vertici che viene infine restituita. Tale operazione ha una complessità temporale di $O(m)$, dove m è il numero totale di scompartimenti della tabella.
- Metodo `contaArchi()`: Calcola il numero totale degli archi $|E|$ orientati. L'algoritmo scorre nuovamente le celle della Tabella Hash e per ogni singolo nodo memorizzato al loro interno, accede alla rispettiva lista di adiacenza, ne trova la dimensione (nuovamente tramite `.size()`) e, analogamente al metodo `contaNodi()`, somma questi valori in una variabile temporanea `count` inizializzata a 0. Al termine dell'iterazione, la funzione restituisce il valore di `count`. La complessità temporale dell'operazione è $O(m + |V|)$, in quanto la funzione ispeziona ogni cella e visita ogni nodo allocato esattamente una volta.

Nel `main`, i due metodi vengono invocati subito dopo la creazione del grafo; i valori restituiti vengono salvati in due variabili intere dedicate (una per i nodi e una per gli archi) per poi essere stampati a schermo.

Conteggio della distribuzione delle frequenze degli archi: La distribuzione delle frequenze viene calcolata tramite un approccio a due passate in un metodo pubblico dedicato. L'algoritmo procede nel seguente modo:

- **Ricerca del Massimo:** L'algoritmo ispeziona l'intera Tabella Hash (scandendo gli m scompartimenti, i nodi e le relative liste di adiacenza) per individuare il valore della frequenza massima (`max_freq`) presente nel grafo. Se al termine di questa scansione il massimo risulta pari a zero, significa che il grafo è privo di archi e la funzione termina immediatamente l'esecuzione.
- **Allocazione e Conteggio:** Noto il valore massimo, viene allocato un vettore ausiliario di interi (`occorrenze`) di dimensione `max_freq + 1`, inizializzato interamente a zero. Successivamente, scorrendo nuovamente il grafo dall'inizio, la frequenza di ogni singolo arco viene utilizzata direttamente come indice per accedere al vettore e incrementare il relativo contatore in tempo costante (`occorrenze[arco.frequenza]++`).
Nota: Il vettore è dimensionato a `max_freq + 1` poiché ciò garantisce l'accesso sicuro alla cella `occorrenze[max_freq]` necessaria per conteggiare gli archi di peso massimo.
- **Stampa dei Risultati:** Al termine del conteggio, il vettore `occorrenze` viene iterato partendo dall'indice 1 fino a `max_freq`. Vengono stampate a schermo esclusivamente le frequenze il cui contatore è strettamente maggiore di zero.

Questo metodo evita l'uso di strutture dati accessorie più complesse e, nonostante possa comportare un parziale spreco di memoria a causa dei contatori a zero per le frequenze non presenti, privilegia l'efficienza temporale. Grazie al tempo di accesso costante in memoria, garantisce una complessità asintotica di $O(m + |V| + |E|)$, proporzionale unicamente alla doppia esplorazione della Tabella Hash e delle relative liste di adiacenza.

Nel `main`, il metodo viene invocato subito dopo aver stampato a schermo il numero dei nodi e degli archi.

Calcolo del costo ottimo di un cammino Minimax: Il valore del costo ottimo coincide con il peso del cammino minimo associato al nodo destinazione d al termine dell'esecuzione dell'algoritmo di Dijkstra. Perciò all'interno del metodo `calcolaCamminoMinimax`,

prima di procedere alla ricostruzione del vettore, tale valore viene estratto in tempo costante $O(1)$ dalla Tabella Hash di tracciamento tramite l'istruzione

```
int costo_ottimo = taccuino.get_info(nodo_destinazione).costo;
```

Per rendere disponibile questo dato all'esterno, la funzione `calcolaCamminoMinimax` è stata modificata per restituire un oggetto `pair<vector<int>, int>`. All'interno di questa struttura, il primo elemento rappresenta il vettore contenente il percorso finale ricostruito, mentre il secondo elemento trasporta il valore del costo ottimo. Di conseguenza, all'interno del `main`, dopo aver invocato la funzione, i due risultati vengono facilmente separati e salvati in variabili dedicate accedendo rispettivamente ai campi `.first` (per il vettore) e `.second` (per il costo). Infine, a condizione che l'algoritmo abbia effettivamente individuato un cammino valido, il costo ottimo viene stampato a schermo subito dopo la sequenza dei nodi del percorso.

Conteggio Cammini a Costo Minimax Ottimo: Dopo aver trovato un percorso ottimale dal nodo sorgente s al nodo destinazione d , vogliamo trovare tutti i cammini da s a d aventi un costo Minimax uguale al valore ottimo calcolato da Dijkstra.

L'algoritmo impiegato si basa su una visita in profondità (DFS, *Depth-First Search*), integrata con un meccanismo di tracciamento degli stati per evitare loop di esplorazione.

Specifiche

1. **Input:** L'identificatore (AS ID) del nodo sorgente s , del nodo destinazione d ed il valore del costo ottimo precedentemente calcolato (fornito come parametro `Costo`).
2. **Output:** Un valore intero che rappresenta il numero totale dei cammini da s a d aventi un costo Minimax pari al costo ottimo.
3. **Specifiche funzionali e Strutture utilizzate:** L'algoritmo (implementato nel `public` della classe `GrafoHashTable`) si divide in due componenti: un metodo di innesco e la funzione ricorsiva vera e propria, supportati da un riadattamento delle strutture in memoria.
 - **Adattamento delle Strutture Dati (TabellaTracciamento):** Per tracciare in modo efficiente lo stato di esplorazione dei nodi, si è scelto di riutilizzare la classe già implementata per l'algoritmo di Dijkstra, evitando di allocare nuove sovrastrutture di memoria. A tal fine, è stata applicata una lieve modifica alla `struct InfoCammino`, introducendo il campo booleano `visitato`. Di conseguenza, è stata aggiornata anche la logica di allocazione dinamica all'interno del metodo `get_info`: ogniqualvolta viene creato e inserito un nuovo nodo nella Tabella Hash, questo campo viene esplicitamente inizializzato a `false` tramite l'istruzione `table[h].push_back({nodo_id, INF, -1, false})`. Questo adattamento permette di sfruttare le prestazioni di accesso in $O(1)$ della tabella per gestire istantaneamente il blocco e lo sblocco dei vertici durante la DFS.
 - **Inizializzazione:** Il processo viene avviato dal metodo pubblico `contaCamminiMinimax`. Questo si occupa di creare localmente una nuova Tabella Hash `taccuino_dfs` di dimensione 100000 (che, grazie alla modifica appena descritta, alloca di default tutti i vertici come "non visitati"), inizializza a zero la variabile `contatore` e chiama la funzione ricorsiva ausiliaria `DFS_ricorsiva` (anch'essa pubblica) passando le variabili per riferimento per non sprecare memoria. Al termine della ricorsione, restituisce il contatore finale.
 - **Caso Base:** All'inizio dell'esecuzione di `DFS_ricorsiva`, l'algoritmo controlla se il `nodo_corrente` coincide con il `nodo_destinazione`. In caso affermativo, la va-

riabile `contatore` viene incrementata e l'istruzione `return` interrompe il ramo di esplorazione corrente, tornando al chiamante.

- **Marcatura del nodo:** Se non si è giunti a destinazione, si accede al taccuino tramite `taccuino_dfs.get_info(nodo_corrente).visitato = true`, marcando il nodo corrente come visitato per evitare loop infiniti durante l'esplorazione del ramo attuale.
- **Esplorazione e Avanzamento Ricorsivo:** Vengono recuperate le informazioni del vertice tramite `get_nodo(nodo_corrente)` e viene avviato un ciclo sulle sue adiacenze. Per ogni arco uscente verso una destinazione v , l'algoritmo verifica due condizioni: il `peso_arco` deve essere minore o uguale al `Costo` ottimo (se fosse maggiore, per la logica della metrica Minimax, l'intero cammino risulterebbe automaticamente più pesante di quello ottimale) e il nodo v non deve essere già stato marcato (`visitato == false`). Se e solo se queste condizioni sono soddisfatte, l'algoritmo avanza in profondità: invoca nuovamente se stesso passando v come nuovo vertice corrente. In questo modo, l'esplorazione "scende" lungo quel ramo, delegando il compito di trovare un percorso da v fino alla destinazione finale.
- **Sblocco del nodo:** Al termine del ciclo sulle adiacenze, il nodo corrente viene esplicitamente smarcato (`visitato = false`). Questa operazione permette al vertice di tornare disponibile per essere attraversato da altri percorsi validi provenienti da diramazioni differenti dell'albero di esplorazione.
- All'interno del `main`, il metodo `grafo.contaCamminiMinimax` viene invocato esclusivamente se l'algoritmo di Dijkstra ha precedentemente individuato un percorso valido da s a d . In caso affermativo, il numero totale dei cammini ottimi restituito dalla funzione viene memorizzato in una variabile intera dedicata, per poi essere direttamente stampato a schermo.

4. **Complessità attesa:** È fondamentale distinguere l'algoritmo impiegato dalla classica visita DFS. La complessità teorica di una DFS standard, che si limita a visitare ogni vertice e ogni arco una sola volta marcandoli definitivamente, ha un tempo lineare di $O(|V| + |E|)$. Tuttavia, il nostro obiettivo è contare tutti i cammini di costo Minimax. Sbloccando i nodi al ritorno dalla ricorsione, l'algoritmo ripercorre più volte le stesse porzioni di Grafo; perciò la complessità scala in base al numero di percorsi possibili, arrivando al caso pessimo esponenziale $O(|V|!)$ per grafi fortemente connessi e densi. Nel caso medio, grazie al taglio dei rami garantito dal vincolo `peso_arco ≤ Costo`, lo spazio di ricerca viene ridotto rendendo il calcolo efficiente nella pratica.

5. **Dipendenze:** Dipende dal Modulo Algoritmico precedente. La ricerca ricorsiva necessita infatti che l'algoritmo di Dijkstra abbia concluso la propria esecuzione per fornire il Costo Minimax.

Tempi di esecuzione degli algoritmi: I tempi di esecuzione sono stati misurati considerando esclusivamente il calcolo degli algoritmi, escludendo i ritardi causati dalla stampa dei risultati a schermo. Per effettuare le misurazioni è stata impiegata la libreria standard `<chrono>`, sfruttando nello specifico l'oggetto `chrono::high_resolution_clock`, che garantisce la massima precisione temporale offerta dal sistema.

La metodologia di campionamento si basa sul calcolo della differenza tra due timestamp:

- **T_start:** acquisito nell'istante immediatamente precedente alla chiamata della funzione algoritmica.
- **T_end:** acquisito nell'esatto istante in cui la funzione termina la sua esecuzione logica.

La differenza temporale netta ($\Delta T = T_{\text{end}} - T_{\text{start}}$) rappresenta il tempo effettivo impiegato. Un accorgimento particolare è stato adottato per il metodo `stampaDistribuzioneFrequenze`: per evitare di includere nella misurazione i pesanti ritardi causati dalle operazioni di stampa sul terminale, la funzione è stata modificata da `void` a `long long`. Al suo interno, il timer viene fatto partire all'inizio e fermato nell'esatto istante in cui la costruzione del vettore delle frequenze risulta conclusa. La differenza temporale viene calcolata e salvata in una variabile locale; successivamente, la funzione procede a stampare i valori a schermo e infine restituisce il tempo calcolato tramite `return`. In questo modo, nel `main` è stato possibile memorizzare l'output temporale in una variabile dedicata per stamparlo isolandolo dai ritardi di output.

In merito alla gestione dell'output nel `main`, la notifica temporale segue due logiche distinte legate al ciclo di vita del programma: i tempi relativi all'inizializzazione globale (caricamento del grafo, conteggio dei nodi, degli archi e calcolo delle frequenze) vengono stampati a schermo una singola volta, immediatamente dopo l'esecuzione delle rispettive funzioni. Al contrario, poiché la ricerca dei cammini è posta all'interno di un ciclo interattivo (che si interrompe solo quando l'utente inserisce -1 come nodo sorgente), i tempi dell'algoritmo di Dijkstra e della successiva visita DFS vengono calcolati e mostrati iterativamente per ogni coppia di nodi inserita in input. I tempi vengono mostrati a schermo solo ed esclusivamente se è stato effettivamente individuato un percorso valido tra il nodo sorgente e la destinazione.

Per fornire risultati chiari e leggibili, il dato finale viene convertito (tramite `chrono::duration_cast`) utilizzando due diverse unità di misura a seconda del contesto:

- **Millisecondi** (*ms*) per la fase di caricamento e costruzione del Grafo, poiché le operazioni di lettura da disco richiedono tempi nettamente superiori rispetto ai calcoli fatti nella memoria RAM.
- **Microsecondi** (*μs*) per l'esecuzione degli algoritmi in memoria (Dijkstra, DFS e funzioni di conteggio), al fine di catturare con la massima precisione le elevate velocità di calcolo.